

ICS/OT Protocol Analysis — Instructor Answer Key

Work these exercises with the two teaching captures open in Wireshark (`dnp3_substation.pcap`, `mqtt_iiot_telemetry.pcap`) and the Docker lab running. Each exercise lists the steps, then the question to answer.

Student name / date: _____

DNP3 — DNP3

Capture: `dnp3_substation.pcap`

Q1. Find the control lifecycle

1. Open `dnp3_substation.pcap` in Wireshark.
2. Apply the display filter `dnp3.al.func in {3,4,5}` to isolate every control.
3. Identify the legitimate SELECT–OPERATE pair and the rogue DIRECT OPERATE.

Question. How many control messages are there, and which one lacks a SELECT — and why does that matter?

Answer. Three controls: SELECT (frame 16) and OPERATE (frame 20) from the master, and a DIRECT OPERATE (frame 27) from 10.20.0.66. The DIRECT OPERATE has no SELECT interlock and comes from a non-master IP — it is the injected TRIP that opens the breaker.

Q2. Spot the impostor by address

1. Filter `dnp3` and add columns for `ip.src` and `dnp3.src` (DNP3 link source).
2. Compare the IP source and the DNP3 link source for every control frame.

Question. What is inconsistent about frame 27, and which field did the attacker forge?

Answer. Frame 27's IP source is 10.20.0.66 but its DNP3 link source is 100 — the master's address. The attacker forged the link source to impersonate the master. Base DNP3 has no way to reject this.

Q3. Turn packets into detections with ICSNPP

1. In the lab container run: `zeek -C -r /pcaps/dnp3_substation.pcap icsnpp-dnp3`
2. Open `dnp3_control.log` and `dnp3_objects.log`.

Question. Which single log line is your best alert for the attack, and what field makes it detectable?

Answer. The `dnp3_control.log` line 'DIRECT_OPERATE ... Trip ... Success' whose `source_h` is 10.20.0.66. But note that works here only because the attacker kept his real IP while forging the DNP3 link address; a smarter attacker who also spoofs the master's IP (or hijacks its TCP session) defeats a source-IP rule. The durable detection keys on an invariant — a control with no preceding SELECT, from an unexpected session, or off the master's baseline cadence (see 'Detection under adversarial and operational reality').

Q4. Design the control

1. Re-read security findings D1 and D3.
2. Given a substation you cannot re-flash to add DNP3-SA tomorrow, list compensating controls you can deploy this week.

Question. Name three compensating controls that reduce the frame-27 risk without changing the outstation firmware.

Answer. (1) OT firewall rule allowing TCP/20000 to the outstation only from the master IP; (2) network segmentation / a data diode isolating the control LAN; (3) a Zeek+ICSNPP sensor alarming on controls from non-master sources. DNP3-SA is the durable fix once the device supports it.

MQTT — MQTT

Capture: *mqtt_iot_telemetry.pcap*

Q5. Read a password off the wire

1. Open *mqtt_iot_telemetry.pcap* in Wireshark.
2. Apply `mqtt.msgtype==1` and expand the CONNECT tree on frame 4.

Question. What are the HMI's username and password, and which single control would have prevented you from reading them?

Answer. `hmi_operator` / `Plant!ntel2024`, readable in cleartext. Running MQTT over TLS on port 8883 would have encrypted the entire CONNECT, including the credentials.

Q6. Trace one message to two subscribers

1. Filter `mqtt.msgtype==3` (all PUBLISH).
2. Follow the third telemetry reading from the sensor through the broker.

Question. Starting at the sensor's publish (frame 46), which frames deliver that same reading, and to whom?

Answer. Frame 46 (sensor→broker) is fanned out to the legitimate HMI in frame 48 and to the rogue eavesdropper in frame 50. One publish, two deliveries — the second is an unauthorized leak.

Q7. Catch the anonymous intruder

1. Filter `mqtt.msgtype==1` and compare the connect flags of frames 4, 15, and 38.
2. In the lab container run: `zeek -C -r /pcaps/mqtt_iot_telemetry.pcap` and open `mqtt_connect.log`.

Question. Which CONNECT is anonymous, and what does `mqtt_connect.log` show for it?

Answer. Frame 38 (client `mqtt-explorer-x`) has no username/password — visible in the CONNECT connect-flags. But do NOT try to catch it by 'empty will fields' in `mqtt_connect.log`: that log has no username/password column at all, and the legitimate authenticated HMI (`hmi-scada-01`) also logs empty will fields — so that discriminator both misses and false-positives.

Detect anonymous access from the broker's own auth telemetry (Mosquitto's '... as anonymous' / repeated CONNACK rc=5), not Zeek's connect log, which sees only client_id and connect_status, never credentials.

Q8. Harden the broker

1. In the lab, edit mosquitto.conf: set allow_anonymous false, add a password_file, and an acl_file scoping the HMI to read plant/+/telemetry only.
2. Restart the broker and re-run the publisher/subscriber and a '#' subscriber.

Question. After hardening, what happens to the anonymous connect, and how does the capture differ?

Answer. The anonymous CONNECT is refused with CONNACK return code 5 (Not Authorized) — verified in the lab — so the rogue host never connects or subscribes, and the leaked-telemetry frames disappear. The ACL is defense-in-depth: Mosquitto enforces it per message topic (a client only receives topics its rule grants) rather than by failing the '#' SUBACK, so scope credentials tightly instead of relying on rejecting the wildcard itself. Adding TLS on 8883 further hides credentials and payloads entirely.

Synthesis

Q9.

A classmate reduces both intrusions to one root cause — 'neither protocol authenticates, so the rogue host is trusted.' That is only half right. Using the captures, separate AUTHENTICATION (who are you?) from AUTHORIZATION (are you allowed to do this?). For DNP3 frame 27 and for the MQTT command injection, state (i) whether authentication happened at all, (ii) whether the failure is one of authentication or authorization, and (iii) the exact missing mechanism that would have stopped it. Cite the specific frame(s).

Answer. The classmate conflates two different failures. DNP3 (frame 27): there is NO authentication of any kind — the outstation never checks identity; the only 'identity' is a 16-bit link address (here forged to 100, the master's) that any host can write into a frame, so the rogue at 10.20.0.66 simply opens its own session and issues a DIRECT OPERATE (Trip). Authentication failure. Missing mechanism: DNP3 Secure Authentication (SAv5) — a challenge-response MAC on critical function codes — plus source allow-listing. MQTT (frame 52, the PUBLISH to plant/tank1/command — NOT frame 54, which is the rogue's DISCONNECT): authentication DID occur and SUCCEEDED — the rogue authenticated anonymously at CONNECT (frame 38) and the broker accepted it with CONNACK return code 0 (frame 40). The injection is therefore an AUTHORIZATION failure: with no per-topic write ACL, an accepted client may publish to a command topic it should never write. Missing mechanism: a broker write-ACL scoping publish rights per client/topic (and, upstream, allow_anonymous false so the anonymous identity is refused before any topic write). Bottom line: DNP3 fails because identity is never checked; MQTT fails because an accepted identity is never constrained — 'no authentication' describes DNP3, while MQTT's gap is 'no authorization,' and the fixes differ (DNP3-SA vs. broker ACLs).

Q10.

For each protocol, name the standard/control that adds authentication and say whether it also adds confidentiality.

Answer. DNP3: Secure Authentication (SAv5 / IEEE 1815-2012) adds authentication + integrity of critical functions but NOT confidentiality (use TLS/VPN for that). MQTT: TLS on 8883 adds confidentiality + protects credentials; authorization still depends on broker-side ACLs, and authentication on broker credentials/certs.

Q11.

You can monitor but not immediately re-engineer these systems. Give one Zeek/ICSNPP-based detection for each capture.

Answer. DNP3: alert on any dnp3_control.log control (SELECT/OPERATE/DIRECT_OPERATE) whose source host is not the sanctioned master. MQTT: alert on mqtt_connect.log connects with empty credentials/anonymous, or mqtt_subscribe.log subscriptions to '#'.